Cluster Analysis

Unsupervised Learning

Previously in this course we have focused on **supervised learning**. In supervised learning, we have predictors X and and a response Y and we are interested in the relationship between the two. It is called "supervised" because we a labeled response variable in the training data and it is predicting that response that "supervises" the model fitting.

In contrast, there is no response variable in **unsupervised learning**. Instead, we are looking for relationships between samples or relationships between predictors without reference to a response. The three common categories of unsupervised learning are **clustering**, **association rules**, and **dimension reduction**.

Cluster analysis: Cluster analysis refers to a large family of methods that attempt to sort objects (e.g. sampling units) into discrete groups called clusters. Ideally, objects within the clusters are more similar in some sense than objects between groups. The key difference between ordination and clustering is that ordination seeks to arrange objects along a continuum and clustering seeks to place objects in discrete groups.

Association rules: The goal of association rule analysis is to find interesting "if-then" relationships between items in a data set. It is often called **market basket analysis** because it is frequently used to identify products that are purchased together. For example, a person going to the grocery store on a Saturday to buy ingredients for making pancakes might buy flour, eggs, and milk. Thus, an association rule might be $(flour, eggs) \rightarrow (milk)$). The goal is to find relationships of this nature. We will not discuss association rules in this course.

Dimension reduction: The goal of dimension reduction is to reduce higher dimensional data to lower dimensional data while preserving essential information. The key idea is that there is often redundant information across variables and so we don't need to keep all of them. **Principle components analysis** is the most common dimension reduction technique.

Clustering

Cluster analysis refers to a large family of methods that attempt to sort objects (e.g. sampling units) into discrete groups called clusters. Ideally, objects within the clusters are more similar in some sense than objects between groups.

There are many different types of cluster analysis. Some of the key differences:

- How is similarity defined between two objects, between two groups, and between an object and a group?
- Do we start with each object in its own group and then aggregate, or do we start with all of the ojects in one group and then disaggregate?
- Is aggregation between groups based on the average similarity between members of tbe groups or on the maximum or minimum similarity between members of the groups?

Definition of a cluster:

There have been several attempts to define "cluster". The most intuitive is as follows: Suppose we have n sampling units. We have measurements of p variables for each sampling unit. Then, we can view the data as a cloud of n points in a p-dimensional space. Clusters are continuous regions with a relatively high density of points separated by regions with a relatively low density of points.

Here is a clustering of clustering methods:

A Clustering of Clustering Methods

Optimal partitioning methods (kmeans, pam , clara, fanny)
- Produces a clustering for fixed number of clusters K.
- Need an initial clustering.
- Lots of different criteria to optimize, some based on probability models.
- Can have distinct outlier group(s).

Agglomerative hierarchical methods (hclust, agnes, mclust)
- Produces a set of clusterings, usually one with k clusters for each $k = n, ..., 2$, successively amalgamating groups.
- Main differences are in calculating group-group dissimilarities from point-point dissimilarities.
- Computationally easy.

Divisive hierarchical methods (diana, mona)
- Produces a set of clusterings, usually one for each k = 2, . . . , K « n .
- Computationally nigh-impossible to find optimal divisions (Gordon, 1999)
- Most available methods are monothetic (split on one variable at each stage).

Partitioning Methods

The simplest partitioning method is the k-means algorithm. The user prespecifies a number k of clusters. The algorithm then chooses k cluster centers that minimize the the within-group sum-of-squares.

The basic algorithm is as follows:

Suppose that we have n sample feature vectors $x_1, x_2, ..., x_n$ all from the same class, and we know that they fall into k compact clusters, $k < n$. Let m_i be the mean of the vectors in cluster i. If the clusters are well separated, we can use a minimum-distance classifier to separate them. That is, we can say that x is in cluster i if $||x - m_i||$ is the minimum of all the k distances. This suggests the following procedure for finding the k means:

Make initial guesss for the means $m_1, m_2, ...m_k$
Until there are no changes in the any mean
{ Use the estimates means to classify the samples into clusters
    For i **in from 1 to** k
    { Replace m_i **with the mean of all samples from cluster** i
    }
}

This is intended for quantitative variables and is most appropriate for continuous variables.

Simulated example

We will start with an example with simulated data. We will use the same simulated data as we did in the classification notes. This is 2D data from a mixture of three bivariate normal distributions.

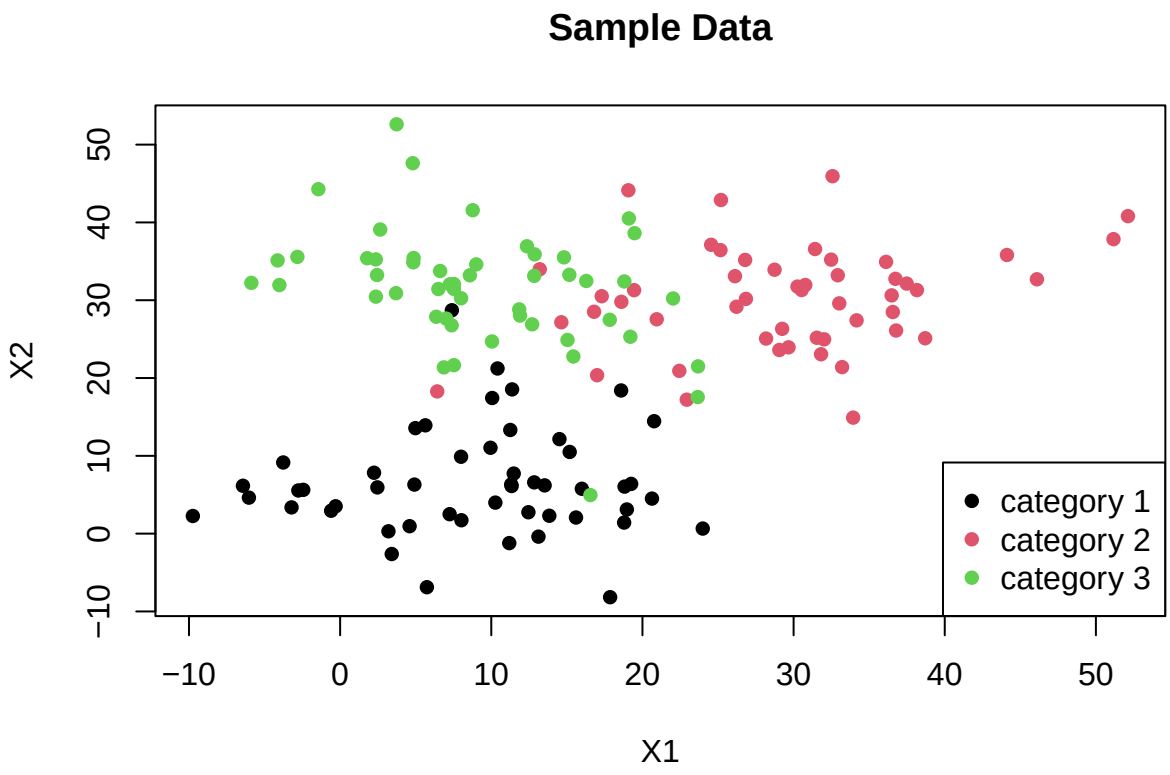
```
library(MASS) #need for mvrnorm function
set.seed(1234)
#sample sizes for the three categories 1, 2, and 3:
n=50
n1=n
n2=n
n3=n
#means for the three categories
mu_x1=10
```

```
mu_y1=10
mu_x2=30
mu_y2=30
mu_x3=10
mu_y3=30

sd0=60  #SD

#generate values for sample 1:
d1=mvrnorm(n1,mu=c(mu_x1,mu_y1),Sigma=sd0*diag(2))
d2=mvrnorm(n2,mu=c(mu_x2,mu_y2),Sigma=sd0*diag(2))
d3=mvrnorm(n3,mu=c(mu_x3,mu_y3),Sigma=sd0*diag(2))

simdat=rbind(d1,d2,d3)
simdat=cbind(simdat,c(rep(1,n1),rep(2,n2),rep(3,n3)))
rownames(simdat)=1:150
plot(simdat[,1:2],col=simdat[,3],xlab="X1",ylab="X2",main="Sample Data",pch=16)
legend("bottomright",c("category 1","category 2","category 3"),col=1:3,pch=16)
```



Now we apply the k-means algorithm to the X data only. The R function for doing this is `kmeans`:

```
?kmeans
```

The `centers` argument to kmeans specifies how many clusters there should and how the initial cluster centers are chosen. Either the number of clusters or a set of initial (distinct) cluster centers. If a number, a random set of (distinct) rows in x is chosen as the initial centers.

Apply to the X data without the class:

```
set.seed(23423)
kmclust=kmeans(simdat[,1:2],centers=3)
kmclust
```

```
## K-means clustering with 3 clusters of sizes 44, 50, 56
##
## Cluster means:
##        [,1]      [,2]
## 1 31.878779 29.840453
## 2  9.022704  5.956537
## 3  9.460280 32.197243
##
## Clustering vector:
##   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
##   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   3
##  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
##   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
##  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
##   3   2   2   2   2   2   2   2   2   2   1   1   3   1   1   3   1   1   1   1
##  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
##   1   1   1   1   1   1   3   1   1   3   3   1   1   1   3   1   1   2   1   1
##  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
##   1   1   1   1   1   3   1   1   1   1   1   1   1   1   3   1   1   1   1   1
## 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
##   3   3   3   3   3   3   3   3   3   3   3   3   3   1   3   3   3   3   3   3
## 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
##   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   2   3   1   3
## 141 142 143 144 145 146 147 148 149 150
##   3   3   1   3   3   3   3   3   3   3
##
## Within cluster sum of squares by cluster:
## [1] 4156.270 4941.219 4749.005
##  (between_SS / total_SS =  72.9 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

The main output is the set of cluster assignments. There is also the within-cluster sum of squares, cluster means, and several other components.

We see that the cluster centers come quite close to the true centers:

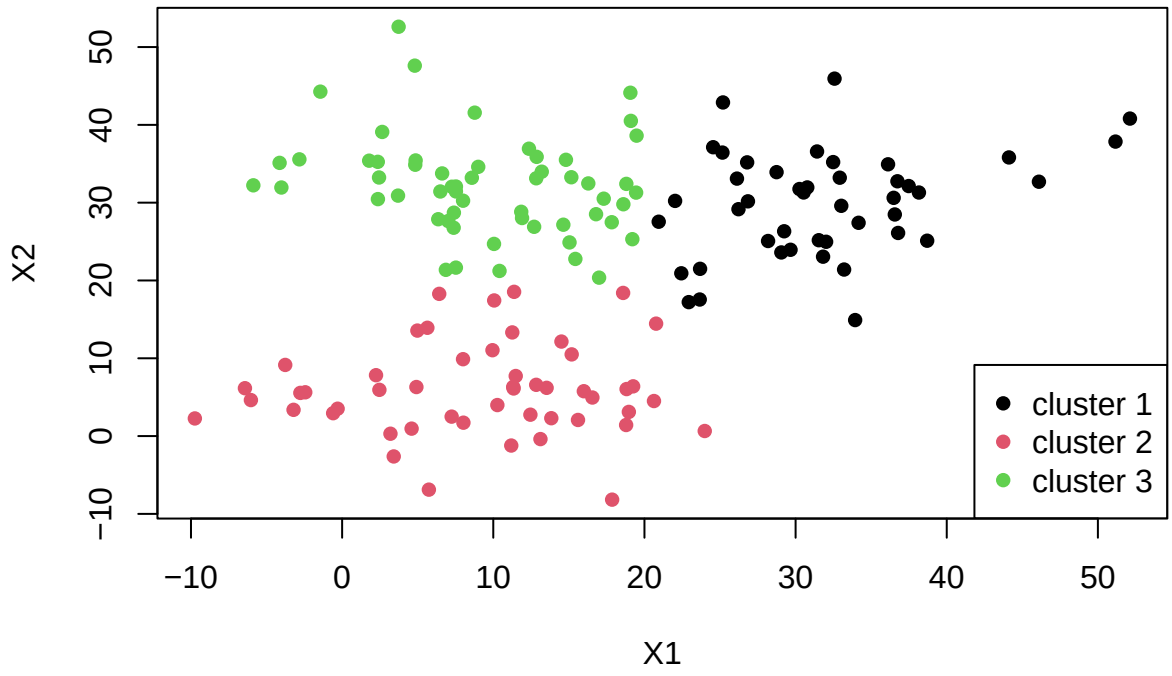
```
kmclust$centers
```

```
##        [,1]      [,2]
## 1 31.878779 29.840453
## 2  9.022704  5.956537
## 3  9.460280 32.197243
```

Keep in mind that we did not give `kmeans` the information about class, it is working with only the positions of the points. Since the positions are generated in relatively compact clusters, the k-means algorithm works well.

This plot shows the cluster assignments:

```
plot(simdat[,1:2],xlab="X1",ylab="X2",main="",col=kmclust$cluster,pch=16)
legend("bottomright",c("cluster 1","cluster 2","cluster 3"),col=1:3,pch=16)
```




```
#plot cluster centers and classes:
#plot(simdat[,1:2],col=simdat[,3],xlab="X1",ylab="X2",main="",pch=kmclust$cluster)
#legend("bottomright",c("category 1","category 2","category 3"),col=1:3,pch=16)
#legend("right",c("cluster 1","cluster 2","cluster 3"),col=1,pch=1:3)
```

Comparing with the previous plot, we see that the cluster assignments correspond closely to the classes. We would likely use classification methods if we had the class variable. If we didn't have the class variable and instead were looking for patterns in the data without the benefit of the class to guide us, we would likely use clustering.

Note that I set the random number seed before running `kmeans`. This is because when we specify the number of clusters and not the cluster positions, the function randomly chooses lines of the data to be cluster centers. This means that the resulting clusters can vary. Because this data is easy to cluster, we get the same clusters most of the time. However, if you run it repeatedly you will occasionally get bad clusters.

#Exercise Take out the random seed and rerun the code for clustering and plotting the clusters until you get clusters that match the true simulated clusters poorly.

Clustering with Breast Cancer Data

We will now see the use clustering with gene expression data for breast cancer.

Here is the data:

```
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## x dplyr::select() masks MASS::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
bcdat=read_csv("../../../../Data/HedenfalkBreastCancer/Hedenfalk.csv",skip=2)
```

```
## Rows: 3226 Columns: 25
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (2): PlatePosition, Title
## dbl (23): ImageCloneID, s1996, s1822, s1714, s1224, s1252, s1510, s1900, s17...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
bcdat
```

```
## # A tibble: 3,226 x 25
##    PlatePosition ImageCloneID Title    s1996 s1822 s1714 s1224 s1252 s1510 s1900
##    <chr>                <dbl> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
##  1 HK1A1                21652 catenin~  0.15  0.22  0.3   0.26  1.22  0.44  0.35
##  2 HK1A2                22012 ADP-rib~  1.54  1.27  0.76  0.85  1.27  0.64  0.9
##  3 HK1A4                22293 uroporp~  1.72  1.57  2.13  1.09  1.98  0.74  1.71
##  4 HK1A5                22493 ribosom~  0.71  1.24  1.69  2.23  1.16  0.82  1.44
##  5 HK1A6                23019 guanine~  0.94  1.53  1.87  1.19  1.16  1.54  1.05
##  6 HK1A7                23132 pre-mRN~  0.8   0.95  1.53  1.37  1.02  1.22  0.78
##  7 HK1A8                24145 adenyly~  0.78  0.81  1.42  0.97  1.04  1.12  1.04
##  8 HK1A9                25584 ubiquin~  0.27  0.35  0.87  0.88  0.51  0.45  0.35
##  9 HK1A10               25725 farnesy~  1.84  1.56  0.85  0.94  0.98  1.1   0.68
## 10 HK1A11               26184 phospho~  0.82  1.2   0.74  1.08  0.73  1.27  0.64
## # i 3,216 more rows
## # i 15 more variables: s1787 <dbl>, s1721 <dbl>, s1486 <dbl>, s1572 <dbl>,
## #   s1324 <dbl>, s1649 <dbl>, s1320 <dbl>, s1542 <dbl>, s1281 <dbl>,
## #   s1321 <dbl>, s1905 <dbl>, s1816 <dbl>, s1616 <dbl>, s1063 <dbl>,
## #   s1936 <dbl>
```

"Gene-Expression Profiles in Hereditary Breast Cancer" by Hedenfalk et al (2001).

Description of Data In this study, RNA from samples of primary breast cancer tumors from seven carriers of the BRCA1 mutation, eight carriers of the BRCA2 mutation, and seven patients with sporadic cases of breast cancer was compared using a microarray of 6512 complementary DNA clones of 5361 genes.

BRCA1 and BRCA2 mutations are explained at this link: https://www.cancer.gov/about-cancer/causes-prevention/genetics/brca-fact-sheet

This is an example of gene expression data. Each row corresponds to a different gene. The first three columns give information about the gene. Columns 4 through 25 correspond to the 22 sample individuals and give the expression values for each gene in each individual. The data is in the form of a ratio of expression intensity.

Gene expression ratios included in the data file were derived from the fluorescent intensity (proportional to the gene expression level) from a tumor sample (BRCA1, BRCA2,or Sporadic) divided by the fluorescent intensity from a common reference sample (MCF-10A cell line). The common reference sample is used for all 21 microarray experiments. The goal of this study was to identify patterns of gene expression that differ between the tumor types. This could potentially aid in diagnosis and treatment.

Data Preprocessing We will first log-transform the data because gene expression data is usually right skewed. I will also an index for the genes because the gene names are often long.

```
bcdat=mutate(bcdat,across(where(is.numeric),log))
#bcdat=mutate(bcdat,gene_index=1:nrow(bcdat),.before=PlatePosition)
```

Also, note that the data has rows corresponding to genes add columns corresponding to samples. This is not "tidy" format - we want rows to correspond to samples and columns to the variables (gene expression) being measured.

I next will rearrange the data into a data from with rows corresponding to samples and columns to genes. I also add in columns for the sample ID and the tumor type. I add column names for the genes consisting of a gene index.

```
bcdatall=read_csv("../../../../Data/HedenfalkBreastCancer/Hedenfalk.csv") #data including extra header
```

```
## New names:
## * `` -> `...2`
## * `` -> `...3`
## * `10` -> `10...10`
## * `10` -> `10...13`
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 3228 Columns: 25
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (24): NEJM-PatientID, ...3, 1, 5, 3, 7, 2, 4, 10...10, 9, 8, 10...13, 16...
## dbl  (1): ...2
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#row 2 has sample id info
#row 3 has tumor info

GEdat=t(bcdat[,-(1:3)])
colnames(GEdat)=paste("g",1:ncol(GEdat),sep="")  #add gene index


GEdat=as_tibble(GEdat)

GEdat=mutate(GEdat,sample=as.character(bcdatall[2,])[-(1:3)],.before=1)
GEdat=mutate(GEdat,tumor=as.character(bcdatall[1,])[-(1:3)],.after=sample)
GEdat$tumor[17]="BRCA1" #paper ided this samples as mislabeled from expression data
```

Rather than examine all genes, we will focus on those that have the biggest difference between BRCA1 and BRCA2. We will do this by conducting t-tests for each gene:

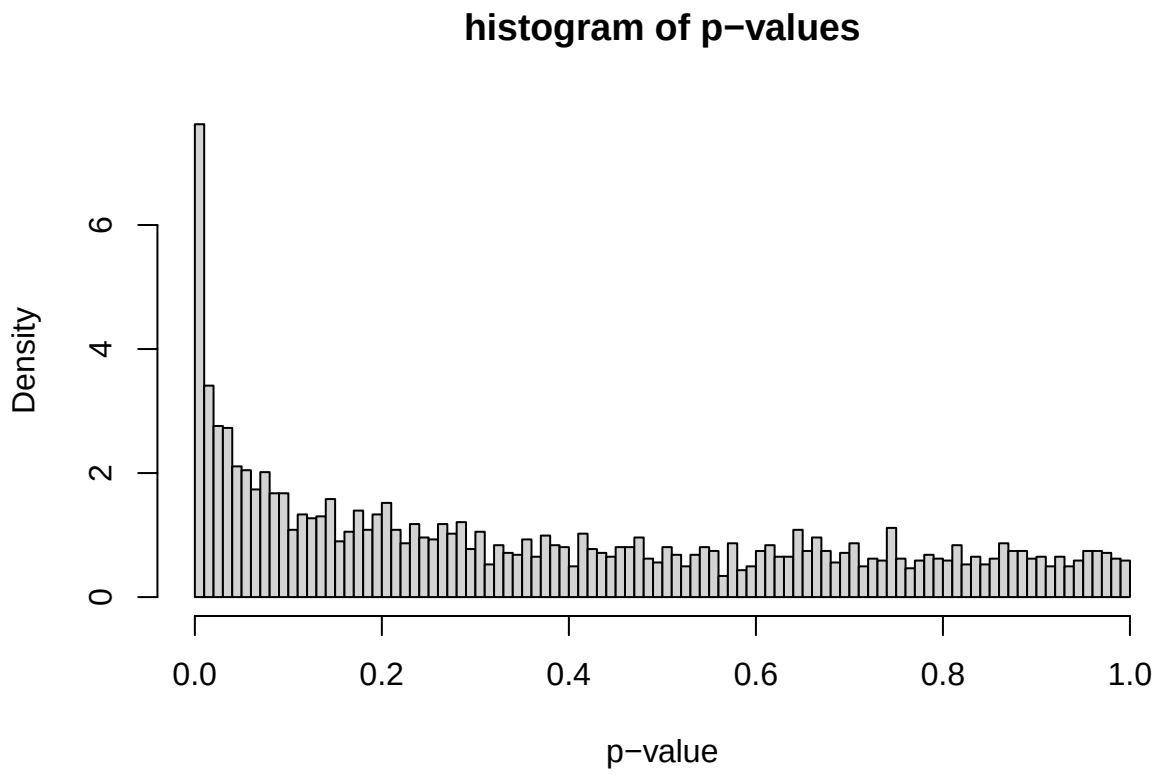
```
 dat1=filter(GEdat,tumor=="BRCA1")
 dat2=filter(GEdat,tumor=="BRCA2")



tval<-NULL
pval<-NULL
for(x in 1:(ncol(GEdat)-2))
{

  pval<-c(pval,t.test(dat1[,2+x],dat2[,2+x])$p.value)
    tval<-c(tval,t.test(dat1[,2+x],dat2[,2+x])$statistic)

}
hist(pval,xlab="p-value",main="histogram of p-values",prob=T,breaks=100)
```




```
sum(pval<0.05)
```

```
## [1] 601
```

```
sum(pval<0.05/ncol(GEdat))
```

```
## [1] 2
```

Only two genes are significant at the string Bonferroni-corrected level, but many are likely associated with breast cancer. We will arbitrarily take the 100 genes that are show the most evidence for differences between BRCA1 and BRCA2:

```r
brgenes<-order(pval,decreasing=F)[1:100]

GEdatFinal=GEdat[,c(1:2,2+brgenes)]
```

```r
dat1=filter(GEdatFinal,tumor=="BRCA1")
dat2=filter(GEdatFinal,tumor=="BRCA2")

t.test(dat1[,4],dat2[,4])
```

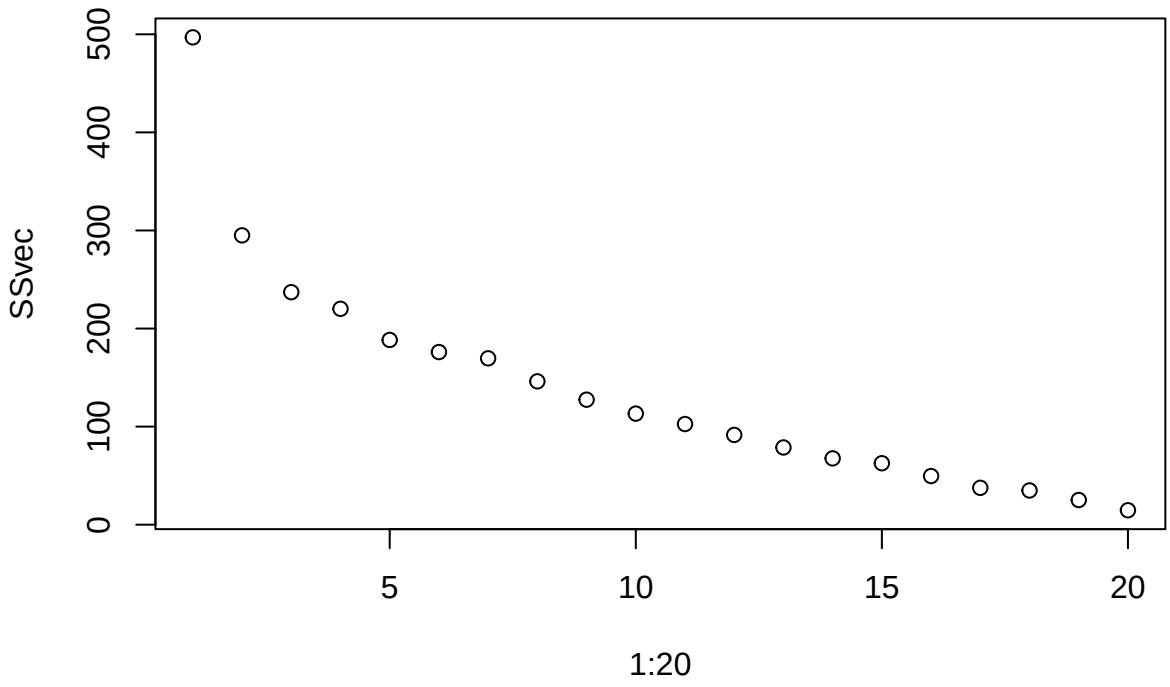
```
##
##  Welch Two Sample t-test
##
## data:  dat1[, 4] and dat2[, 4]
## t = 6.7105, df = 12.923, p-value = 1.491e-05
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.6509412 1.2696217
## sample estimates:
##  mean of x  mean of y
##  0.5566331 -0.4036483
```

Choosing the number of clusters Now, we will cluster the samples with this genes. One import question is how many clusters should we use? A common strategy for choosing k is using an **elbow plot**. We make a plot of the within-group sum of squares versus the number of groups. The within-group SS will always decrease as we add more clusters, but we will often see a pattern of rapid decrease followed by a leveling out. The point where the curve begins to level out is called the **eblow** of the curve and we take the value of k at the elbow as optimal. Let's try it

```r
SSvec=NULL
for(k in 1:20)
{
  SSvec=c(SSvec,kmeans(GEdatFinal[,-(1:2)],centers=k)$tot.withinss)


}

plot(1:20,SSvec)
```



We see a sharp drop with k=1 to 3 and then a more steady gradual drop. We will take $k = 3$ is the elbow. Thus, we go with three clusters.

```
bcclusterKmeans=kmeans(GEdatFinal[,-(1:2)],centers=3)
```

Let's compare how the gene expression clusters align with tumor type:

```
cbind(bcclusterKmeans$cluster,GEdatFinal$tumor)
```

```
##       [,1] [,2]
##  [1,] "2"  "BRCA1"
##  [2,] "2"  "BRCA1"
##  [3,] "2"  "BRCA1"
##  [4,] "2"  "BRCA1"
##  [5,] "2"  "BRCA1"
##  [6,] "2"  "BRCA1"
##  [7,] "3"  "BRCA2"
##  [8,] "3"  "BRCA2"
##  [9,] "3"  "BRCA2"
## [10,] "3"  "BRCA2"
## [11,] "1"  "Sporadic"
## [12,] "3"  "Sporadic"
## [13,] "1"  "Sporadic"
## [14,] "1"  "Sporadic"
## [15,] "1"  "Sporadic"
## [16,] "1"  "Sporadic"
## [17,] "2"  "BRCA1"
## [18,] "2"  "BRCA1"
```

```
## [19,] "3"  "BRCA2"
## [20,] "3"  "BRCA2"
## [21,] "3"  "BRCA2"
## [22,] "3"  "BRCA2"
```

```
table(bcclusterKmeans$cluster,GEdatFinal$tumor)
```

```
##
##     BRCA1 BRCA2 Sporadic
##   1     0     0        5
##   2     8     0        0
##   3     0     8        1
```

There is perfect alignment, except for one sample the is "sporadic" but clusters with the BRAC2 group.

It is promising that the groups align well with the cancer types. You may ask what the point of clustering is here if all that it does is return what we already know? The answer is that our interest here is understanding how relationships between genes relate to the tumor types. The k-means algorithm does not give us tools to address that, but hierarchical clustering does.

Exercise #1 Write a function `elowplot(dat,upperK)` to make an elbow plot. It should take input of a data from `dat` and a maximum value `upperK` of `k` and output the elbow plot. Apply it to the simulated data above.

Agglomerative hierarchcal clustering

In **agglomerative hierarchical clustering**, we start with each object in its own cluster. Then, using a similarity or dissimilarity matrix, we successively agglomerate objects and build up a hierarchy. At each stage the two most similar objects are entities, where an entity could either be one of the original objects or a cluster of the those objects. A key issue is how distances are measured between an object and a group or between too groups:

- Complete linkage. This designates the distance between an object and a cluster (or two clusters) as the distance between the most widely separated objects. This is the default for `hclust()`.
- Single linkage designates the distance between an object and a cluster (or two clusters) as the distance between the closest objects.
- Centroid linkage minimized the distance between the centroids ("center of mass")of the clusters.
- Median linkage is similar to centroid linkage except that centroid of newly fused groups is placed at the median of the two old centroids. By contrast, centroid linkage places the new centroid in a fashion weighted by group
- Mean linkage designates the distance between an object and a cluster (or two clusters) as the average distance between them.

Simulation Example

The basic R function for doing hierarchical clustering is `hclust`.

```
?hclust
```

The main input to `hclust` is a matrix of distances between points. The `dist` function can be used to make the matrix of distances:

```
?dist
```

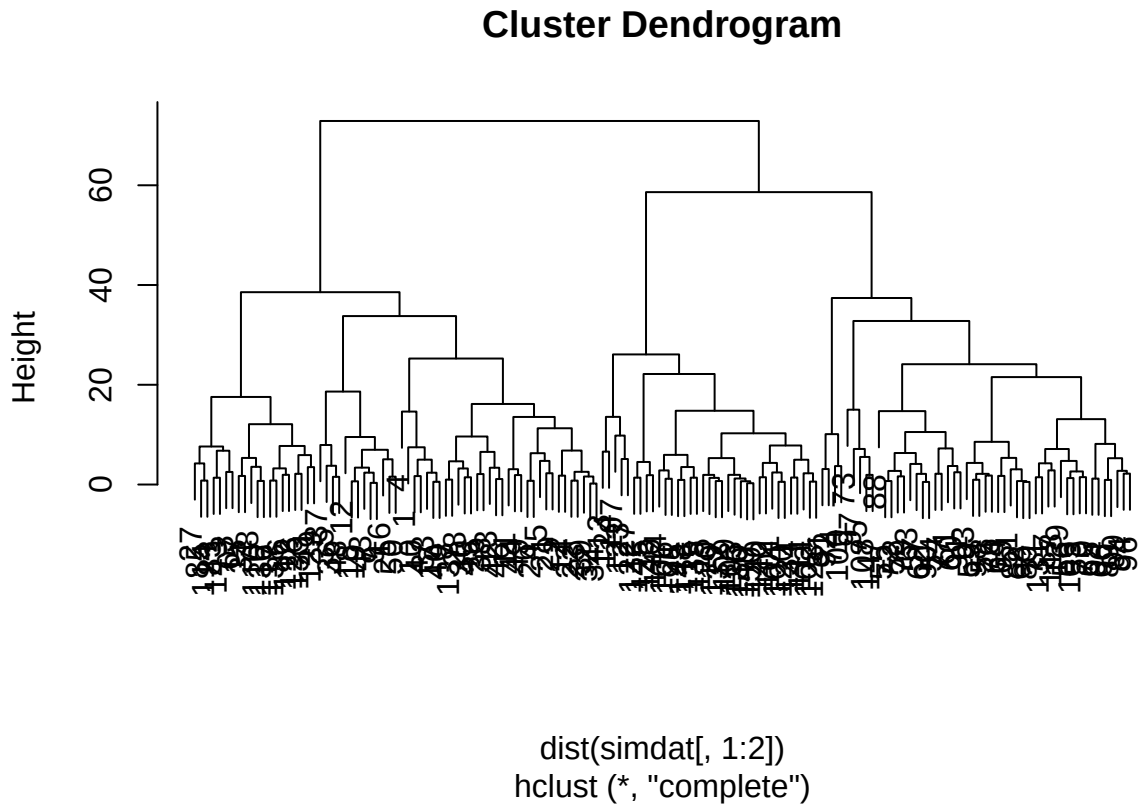
By default, `dist` calculates a Euclidean distance.

```
dist(simdat[1:10,1:2])
```

```
##           1         2         3         4         5         6         7
## 2  14.9032722
## 3  18.5531837  7.4651227
## 4  10.7394784 20.5934727 26.5797010
## 5  17.9648662  3.4572996  8.9178033 22.4855361
## 6  22.6460054  9.0452093 13.7038632 25.2459567  5.6501679
## 7  27.1980591 18.4909982 24.9226596 24.7709740 16.0330359 11.8612668
## 8   9.4950401  6.5528487 12.8987994 14.0606102  8.9178769 13.1775745 18.7555729
## 9  26.8934487 18.1593692 24.6036384 24.5460438 15.7107484 11.5763497  0.3342552
## 10  5.5894510 10.0829978 15.2989419 11.3296535 12.8012688 17.1647054 21.8690879
##           8         9
## 2
## 3
## 4
## 5
## 6
## 7
## 8
## 9  18.4302093
## 10  3.9929944 21.5557106
```

The clustering works on these distances.

```
hclustout=hclust(dist(simdat[,1:2]))
plot(hclustout)
```



The main output of `hclust` is `$merge`:

```
hclustout$merge[1:10,]
```

```
##       [,1] [,2]
##  [1,]  -32  -34
##  [2,] -149 -150
##  [3,]   -7   -9
##  [4,]  -68  -80
##  [5,]  -62  -90
##  [6,] -104 -120
##  [7,]  -17  -40
##  [8,] -102 -128
##  [9,]  -18  -19
## [10,] -130    2
```

This shows the sequence of merges of sites. If the number is negative then it refers to an individual object being merged. For example, -32 -34 means that data points 32 and 34 are being merged. If the number is positive, then it refers to the results of a previous merge. For example, -130 2 means that data point 130 is merged with the second merge above. We see that the second merge was data points 149 and 150.

The `$height` argument gives the value of the criterion used for agolommeration (e.g. the distance). This is given in the same order as the merges. E.g. the distance between data points 32 and 34 (the first merger) is 0.20, the distance between sites 149 and 150 (the second merge) is 0.26, and so on.

```
hclustout$height[1:10]
```

```
##  [1] 0.2009595 0.2685145 0.3342552 0.5216943 0.5229566 0.5521143 0.5565331
##  [8] 0.5890585 0.6454544 0.6599716
```

Note that the y-axis of the plot is giving the height at which the merge occurs. If you look carefully at the plot, you can see that the merger of 32 and 34 is the lowest one on the tree and at occurs at a height of 0.2

The last merge has a height of 72.9. This is the most distantly related merger in the tree and it is highest on the plot.

```
tail(hclustout$height)
```

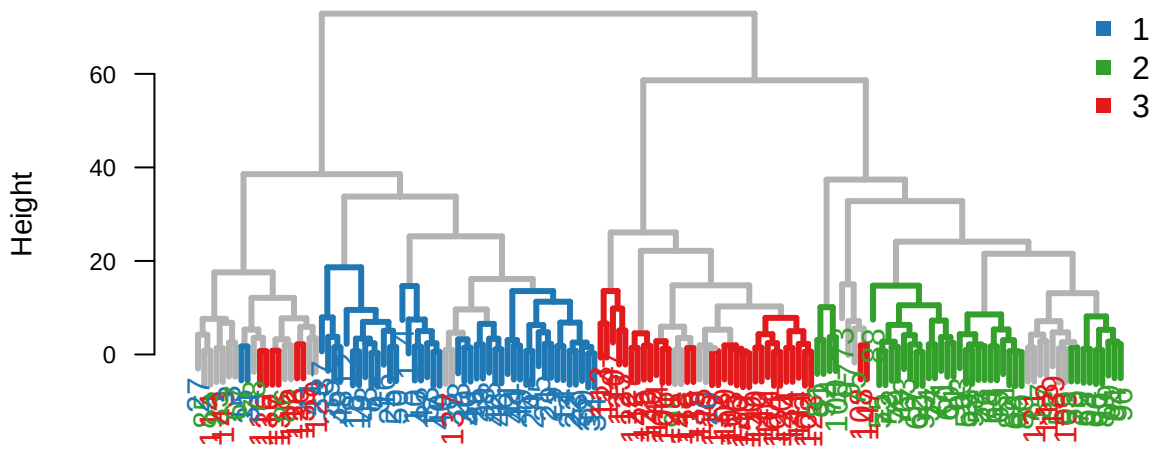
```
## [1] 32.78824 33.77256 37.39907 38.56250 58.62619 72.88779
```

I will use an additional package to add color to the plot to see how the cluster tree relates to the classes that we know exist:

```
library(colorhcplot)
```

```
## Warning: package 'colorhcplot' was built under R version 4.5.3
```

```
colorhcplot(hclustout,factor(simdat[,3]))
```



cutree function The function `cutree` is used to cut a tree above a certain height. We can either specify the desired number of groups (k) or the height at which to cut (h).

```
?cutree
```

For example, the code below This returns a vector of memberships in the three clusters:

```
k3<-cutree(hclustout,k=3)
k3
```

```
##   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   2
##  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
##   1   1   1   1   1   1   1   1   1   1   3   3   3   3   3   3   3   3   3   3
##  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
##   3   3   3   3   3   3   1   3   3   3   3   3   3   3   3   3   3   1   3   3
##  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
##   3   3   1   3   1   1   3   3   3   3   3   3   3   3   2   3   3   3   3   3
## 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
##   2   2   2   2   3   2   2   3   2   1   2   1   2   1   2   2   1   3   2   2
## 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
##   2   2   2   2   2   1   2   2   1   2   3   2   1   2   2   1   1   1   3   2
## 141 142 143 144 145 146 147 148 149 150
##   2   2   1   2   2   2   3   2   2   2
```

You can also give it a vector of k or h values. In this case, it returns a matrix:

```
k1to10=cutree(hclustout,k=1:10)
k1to10[1:10,]
```

```
##    1 2 3 4 5 6 7 8 9 10
## 1  1 1 1 1 1 1 1 1 1  1
## 2  1 1 1 1 1 1 1 1 2  2
## 3  1 1 1 2 2 2 2 2 3  3
## 4  1 1 1 1 1 1 1 1 1  1
## 5  1 1 1 1 1 1 1 1 2  2
## 6  1 1 1 1 1 1 1 1 2  2
## 7  1 1 1 1 1 3 3 3 4  4
## 8  1 1 1 1 1 1 1 1 2  2
## 9  1 1 1 1 1 3 3 3 4  4
## 10 1 1 1 1 1 1 1 1 1  1
```

Column `j` is the group membership when `j` groups are specified. Rows correspond to data points.

```
HCk3<-NULL
for(k in 1:3) #loop over groups
{
    HCk3<-rbind(HCk3,colMeans(simdat[k3==k,1:2]))
}


HCk3
```

```
##          [,1]      [,2]
## [1,] 10.41936  9.890773
## [2,]  6.02042 34.473603
## [3,] 29.91208 31.287521
```

Each row `i` of `HCk3`has the means for each variable for sites that belong to group `i` of `k3`. Note that these values correspond well to the true means of the simulated groups.

If the `members` argument to `hclust` is not `NULL` then the `d` argument is taken to be a dissimilarity matrix between clusters instead of dissimilarities between singletons and `members` gives the number of observations per cluster.

Exercise #2

Plot the data with colors corresponding to the value of `k3`.

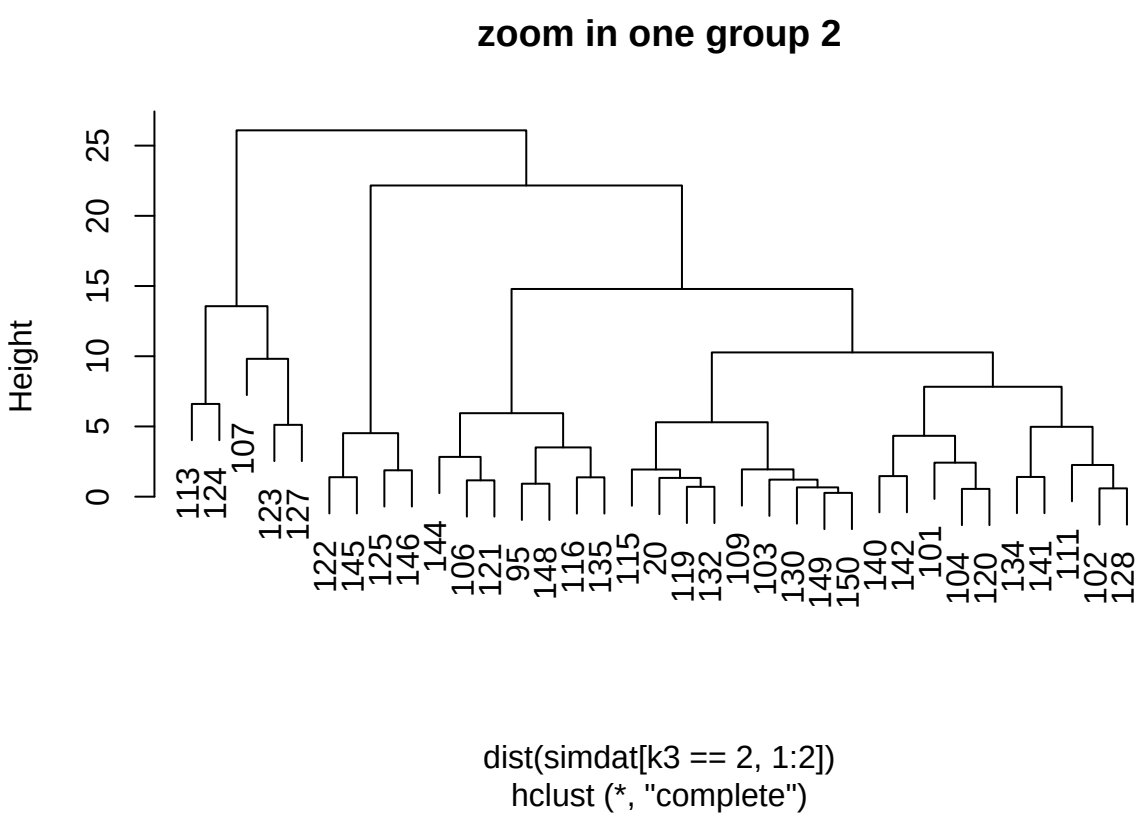
Zooming in on the tree We can also use `cutree` to zoom into a section of the bottom of the tree. For example, lets look at the tree for the sites that got put in group 2:

```
simdat[k3==2,1:2] #this gives only sites that are in group 2
```

```
##          [,1]     [,2]
## 20   7.393693 28.71298
## 95  13.206492 33.97959
## 101  6.615550 33.75855
## 102  1.788316 35.39715
## 103  6.497348 31.43698
## 104  4.862886 35.42786
## 106 12.869861 35.89052
## 107 -1.440557 44.27166
## 109  8.001011 30.25301
```

```
## 111  2.441460 33.23826
## 113  8.771293 41.56855
## 115  7.377246 26.77960
## 116 15.163191 33.26886
## 119  7.018304 27.63964
## 120  4.836697 34.87637
## 121 12.359529 36.93397
## 122 -4.136475 35.11399
## 123  4.805869 47.61033
## 124  2.651924 39.08987
## 125 -5.874607 32.22859
## 127  3.736462 52.61156
## 128  2.357979 35.24724
## 130  7.528631 31.44456
## 132  6.358098 27.87215
## 134  3.697240 30.90508
## 135 16.285316 32.47220
## 140  9.005154 34.60322
## 141  2.366788 30.45806
## 142  8.580571 33.20217
## 144 14.806633 35.50874
## 145 -2.827654 35.56849
## 146 -4.018692 31.94928
## 148 12.840471 33.13300
## 149  7.260829 32.04776
## 150  7.527827 32.07626
```

```
hclust_grp2<-hclust(dist(simdat[k3==2,1:2]))
plot(hclust_grp2, main="zoom in one group 2") #shows a plot of one section of the bottom of the origina
```



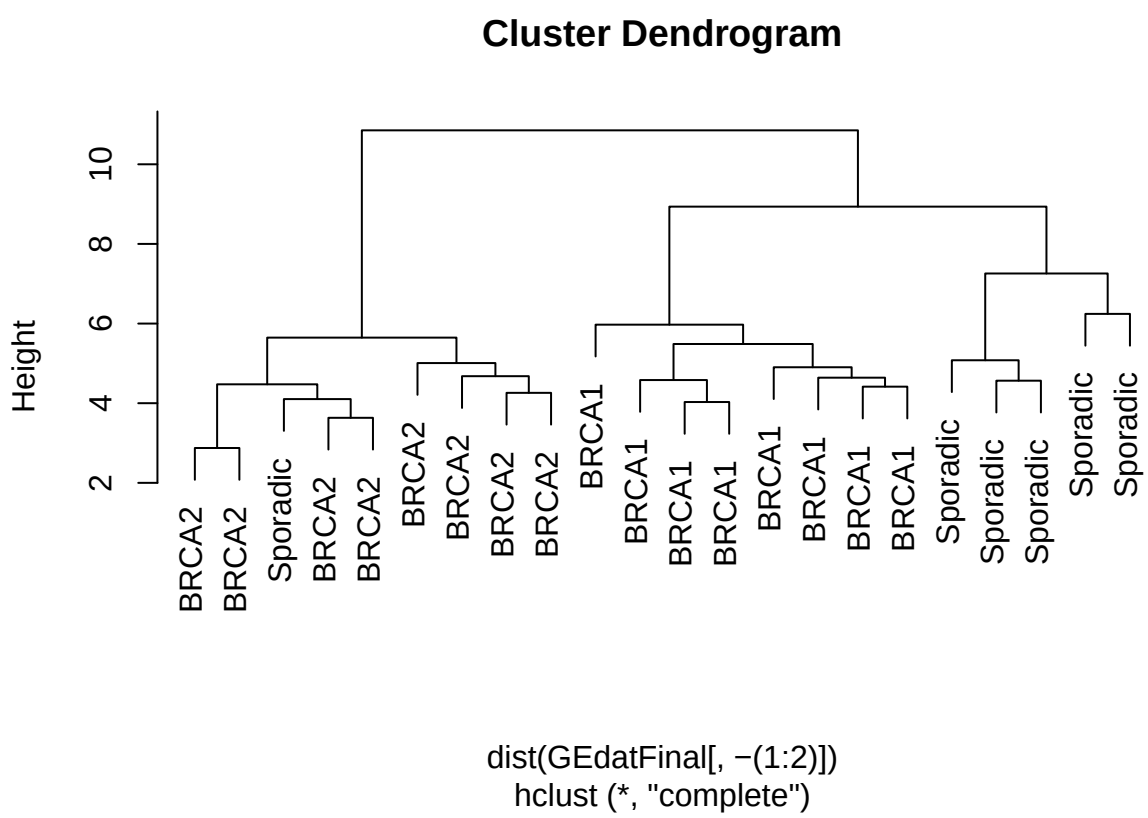
If we examine the above tree, we see that it matches a lower section of the full tree.

Application to Breast Cancer Data

We know return to the breast cancer data.

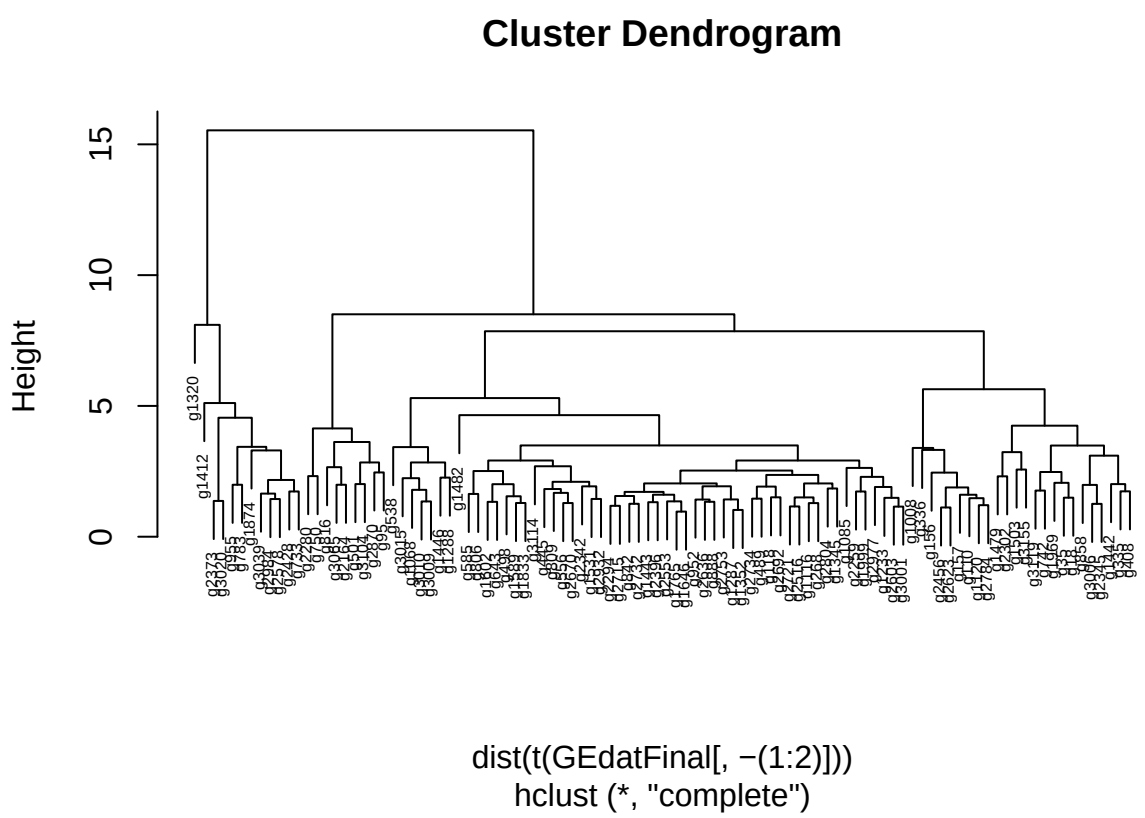
I will apply `hclust` and plot the resulting tree with labels of the tumor type.

```
BCclusterHC=hclust(dist(GEdatFinal[,-(1:2)]))
plot(BCclusterHC,labels=GEdatFinal$tumor)
```



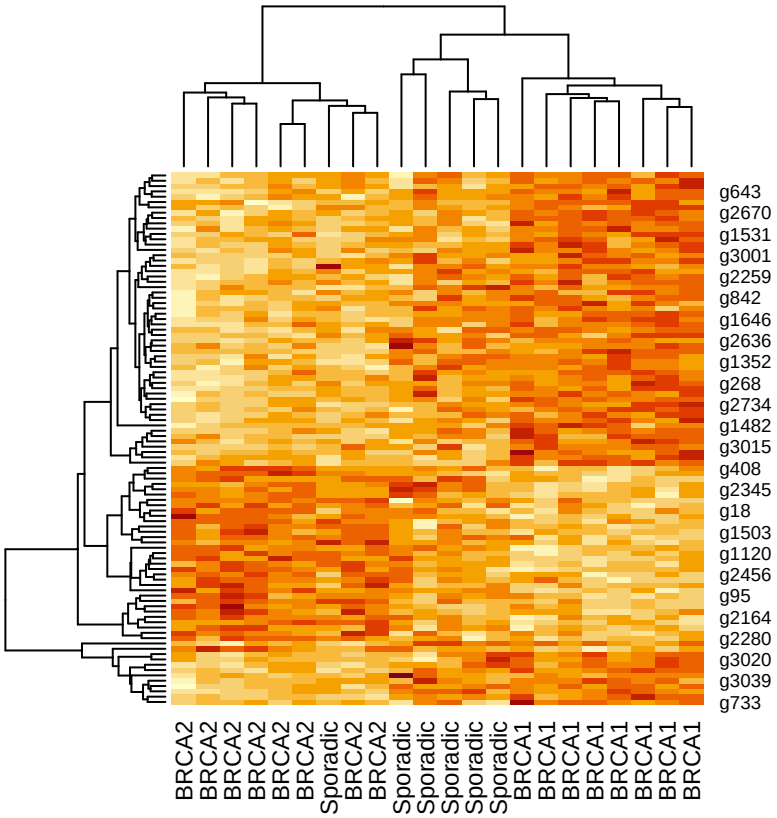
We can also plot cluster the genes:

```
GeneclusterHC=hclust(dist(t(GEdatFinal[,-(1:2)])))
plot(GeneclusterHC,cex=0.5)
```



The most useful thing is to do cluster in both directions using a heatmap:

```
heatmap(as.matrix(t(GEdatFinal[,-(1:2)])),labCol=GEdatFinal$tumor)
```



This does hierarchical clustering in both dimensions (genes on the y-axis and sample individuals on the x-axis). The color corresponds to the intensity of expression for the combination of gene and individual.

We see some clear patterns in the genes. There is an upper group of genes that looks to be down-regulated (has lower expression levels) in BRCA1 than in BRACA2 and possibly sporadics. There is another group of genes (the middle group) that looks upregulated in BRCA1. There is a third group at the bottom that looks upregulated in BRACA2.

These patterns give us potential avenues of research. Were we the researchers, we could identify the genes in these groups and look for commonalities between them. This may point us in the direction of new understandings of breast cancer.

Beware of false patterns

A very large number (hundreds of thousands) of studies of this nature have been published. Many important breakthroughs came out of these studies, also many meaningless dead ends.

Consider the program below. This generates simulated data for two groups (xdat and ydat) for 3000 genes. Both groups are generated from a standard normal distribution and thus there is no difference between them. The program then follows the same procedure as above (that is, select the genes with the 100 lowest p-values and does a heat map)

```
N<-10  #sample size
ngene<-30000  #number of genes
topn=100
xdat<-matrix(nrow=ngene,ncol=N)
ydat<-matrix(nrow=ngene,ncol=N)
```

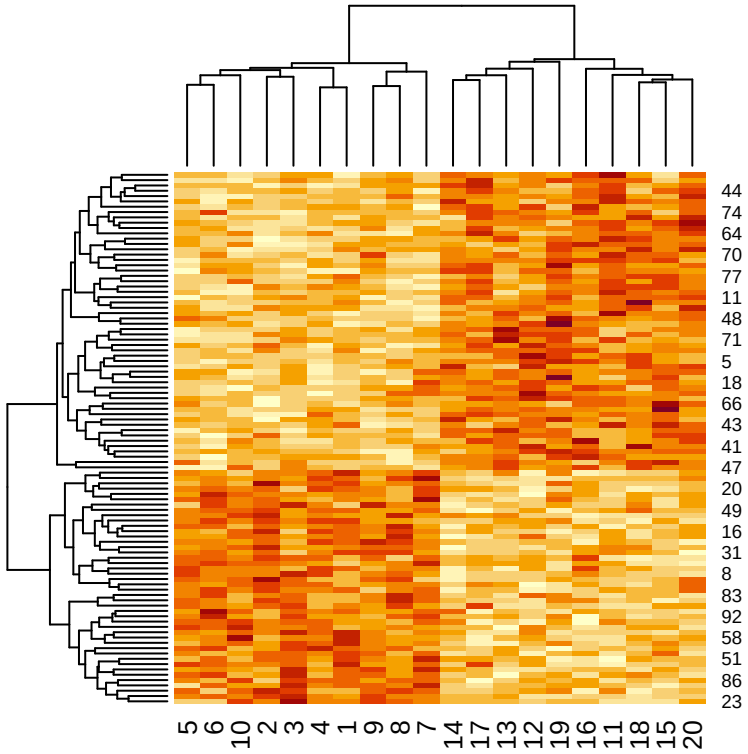
```
xdat[]<-rnorm(N*ngene,mean=0,sd=1)  #randomly generate X data set
ydat[]<-rnorm(N*ngene,mean=0,sd=1)  #randomly generate Y data set
                #both data sets generated from same distn.


#calculate t-statistic for difference between groups for every gene:
tval<-NULL
for(x in 1:ngene)
{   tval<-c(tval,t.test(xdat[x,],ydat[x,])$statistic)

}

brgenes<-order(abs(tval),decreasing=T)[1:topn]  #this gives the row
                #numbers of the 100 genes
                # with the highest absolute value of t-statistic
#brdclust<-hclust(dist(dat2[brgenes,]))  #this does hierarchical clustering
                    #just on the those 100 genes

hm<-heatmap(cbind(xdat[brgenes,],ydat[brgenes,])) #this produces the heatmap for those 100 genes
```



We see that there are clear patterns visible in the heat map despite the fact there are no real patterns in the data (because it was generated randomly). This is because that when you start with 3000 genes, then there will always be some that appear to be have some association with the treatment by chance. The color scale in the heat map is automatically chosen to match the scale of variation of the data and so will always show **something**.

Thus, we can't trust these types of results without furtherer work to verify that the apparent patterns are real and meaningful.

Exercise #3

Repeat the above with 30,000 genes (still taking the top 100 to cluster on). What do you notice about the heat plot? Why do you think this is ?